

Why a Perceptron Fails on Non-Linear Data

Deep Learning Notes · The XOR Problem ♦

1. Quick Recap

- A **Perceptron** is a single neuron. It takes inputs, multiplies them by weights, adds a bias, and passes the sum through a step function.

$$\text{output} = \text{step}(w_1x_1 + w_2x_2 + b)$$

Geometrically, it draws **ONE straight line** to split data into two classes.

2. The Big Limitation

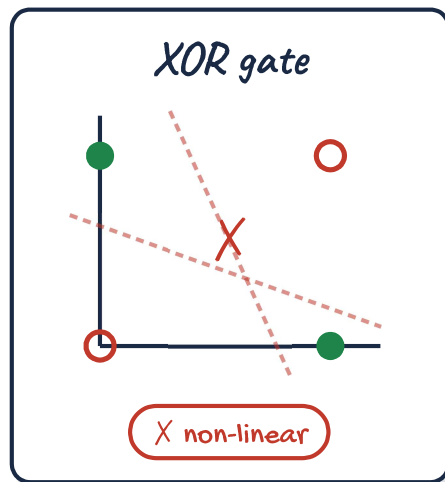
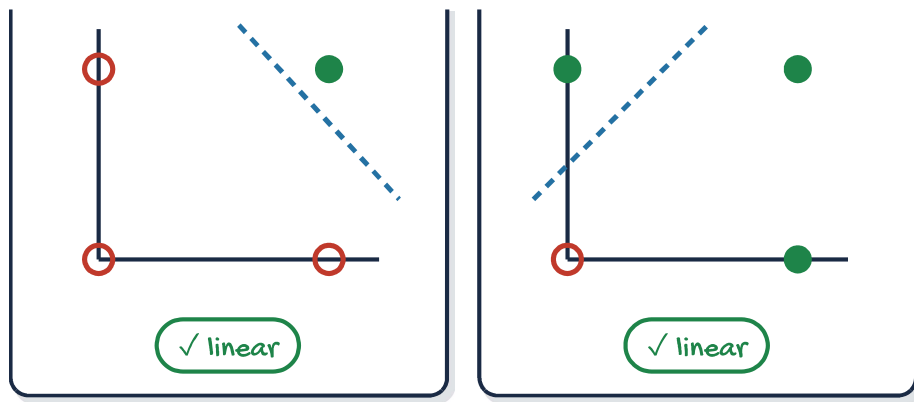
A single Perceptron can **ONLY** separate data that is **linearly separable** — i.e. splittable by 1 straight line.

If the two classes cannot be divided by a single line → the Perceptron **never converges** (weights keep changing forever).

3. Three Test Datasets

AND gate

OR gate



● = output 1 (green filled) ○ = output 0 (red outline) blue dashes = the separating line

— continued on next page —

4. Truth Tables (why XOR is different)

x_1	x_2	AND	x_1	x_2	OR	x_1	x_2	XOR
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

XOR = 1 only when inputs are DIFFERENT → the two 1's sit on opposite corners (a diagonal!)

5. What the Code Showed

- Used *scikit-learn's Perceptron* class on all three.
- AND & OR → separated cleanly by one straight line. ✓
- XOR → no clear boundary; a single line always misclassifies ≥ 1 point. X

TensorFlow Playground: a single neuron solves linear data fast, but on XOR it **never converges** — no matter how many epochs you give it.

6. Why It Never Converges

Training keeps nudging the weights to fix wrong points. On XOR, **every** possible line gets something wrong, so the weights **oscillate forever** and never settle.

7. The Fix → next topic

Stack perceptrons into layers = Multi-Layer Perceptron (MLP)

Hidden layers combine multiple lines → the decision boundary can **bend** and wrap around the XOR pattern. ✓

★ KEY TAKEAWAY ★

A single Perceptron = linear classifier only.

Non-linear problems (like XOR) need **hidden layers**.

◆ Next: Multi-Layer Perceptrons ◆